

C# (.NET): ACCESO A BASES DE DATOS

ADO.NET

Es un conjunto de interfaces, clases, estructuras y enumeraciones que permiten el acceso a los datos desde la plataforma .NET de Microsoft

Proveedor De Datos

Permiten que una aplicación pueda acceder (leer y escribir) a los datos de una fuente de datos. ADO.NET soporta tres proveedores distintos: OLE DB.NET, SQL Server.NET y ODBC.NET.

SQL Server: para base de datos SQL Server 7.0 o posterior

OLE DB: para Oracle, Microsoft JET, SQL Server versiones anteriores a 7.0

ODBC: para cualquier fuente de datos para la que exista un ODBC

Veamos los distintos NameSpaces que hay

- System.Data : clases genéricas de datos de ADO .NET, integra la gran mayoría de clases que habilitan el acceso a los datos de la arquitectura .NET.
- System.Data.SqlClient: clases del proveedor de datos de SQL Server, permiten el acceso a proveedores SQL Server en su versión 7.0 y superior.
- System.Data.OleDb: clases del proveedor de datos de OleDb, permiten el acceso a proveedores .NET que trabajan directamente contra controladores basados en los ActiveX de Microsoft.

Podemos conectarnos a una bbdd MySQL, para ello además de tener instalado MySQL (o el servidor WAMP) debemos instalados los driver mysql-connector-net y mysql-for-visualstudio.

Debemos añadir la referencia Mysql.Data para tener acceso a los objetos definidos en Mysql.Data.MySqlClient

Podemos trabajar con el sql management que incorpora Visual Studio, sin necesidad de instalar nada más de lo que ya tenemos.

Añadir una bbdd sql

1. Abrir orígenes de datos Mayus + alt + D
2. Generar una nueva bbdd
3. En el explorador de servidores actualizo la conexión a esa bbdd para que esté verde.

Añadir tablas

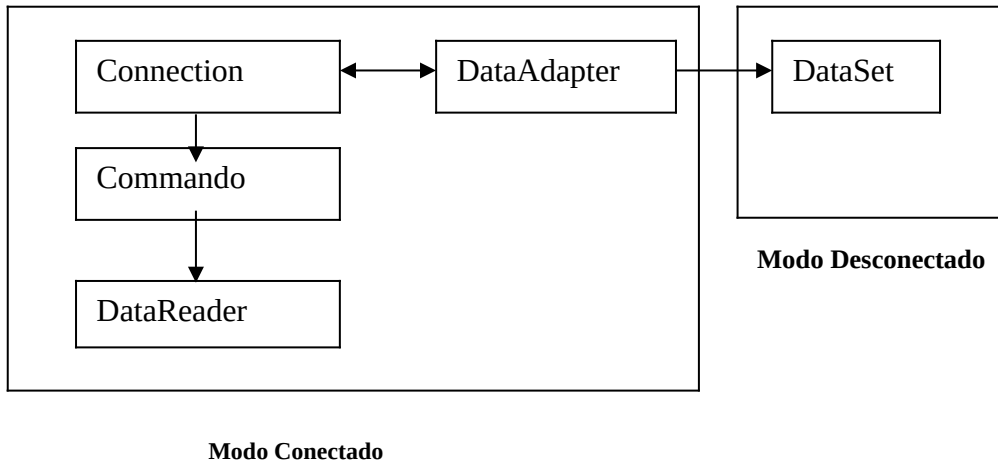
1. Depliego y botón derecho en tabla
2. Añadir tablas
3. Añadir campos
4. En la esquina superior izquierda del **Diseñador de tablas**, selecciona **Actualizar**
5. En el cuadro **de diálogo Vista previa de actualizaciones de base de datos**, selecciona **Actualizar base de datos**.
6. La tabla se crea en el archivo de base de datos local.

Añadir registros

1. Sobre la tabla botón derecho, selecciona **Mostrar datos de tabla** o **Ver datos**.
2. Agregue los datos

Modelo De Objetos

Tenemos dos grupos dependiendo de cómo trabajemos: en modo conectado o sin conexión.



- **Connection:** objeto que permite la conexión con la base de datos
- **Command:** objeto que permite ejecutar los comando SQL para operar con los datos de la base
- **DataReader:** cuando el comando ejecutado obtiene como resultado un conjunto de registros , estos vienen representados por un DataReader
- **DataSet:** registros obtenidos de la base
- **DataAdapter:** puente entre el dataset y la conexión

Estos nombres son genéricos, no los usaremos en código porque cada proveedor tiene su propio NameSpace y sus propios objetos de este tipo:

System.DataDataSet

System.Data.OleDb : OleDbConnection, OleDbCommand, OleDbDataReader

System.Data.SqlClient: SqlConnection,.....

MySQL.Data.MySqlClient: MySqlConnection,...

Cada vez que usemos variables de este tipo debemos importar el NameSpace adecuado

Objeto Connection

Es para abrir la conexión con la base de datos. Tiene una propiedad fundamental **ConnectionString** a través de la cual le decimos qué proveedor vamos a usar y cual es la base de datos que queremos abrir

ConnectionString: Si utilizamos la clase OleDbConnection debemos especificar el proveedor OLEDB que se va a utilizar para establecer la conexión. Los proveedores OLEDB que son compatibles con ADO .NET son:

- SQLOLEDB: Microsoft OLE DB Provider for SQL Server.
- MSDAORA: Microsoft OLE DB Provider for Oracle.
- Microsoft.Jet.OLEDB.4.0: OLE DB Provider for Microsoft Jet.

```
"provider=SQLOLEDB;server=angel;database=bbdd;uid=sa;pwd=");
```

```
"Provider=Microsoft.Jet.OLEDB.4.0;Data Source=bbdd useid=usu;  
password="
```

```
"Server=localhost; Database="nombre bbdd"; Uid="usuario(root)";  
Pwd=""; "
```

La diferencia entre usar una bbdd u otra implica simplemente un cambio del nombre de las clases utilizadas de ADO .NET y la cadena de conexión utilizada para establecerla

La propiedad ConnectionTimeout establece el nº de segundos después de los cuales la conexión falla con error.

Una vez establecida la cadena, se abre con el método open y se cerrará con close. Para comprobar el estado de la conexión utilizamos la propiedad state

```
if (conexion.State ==ConnectionState .Open)
```

Para utilizar el enumerado ConnectionState es necesario importar el namespace System.Data

Para establecer ruta relativa utilizaremos DataDirectory o el objeto Application:

- `conexion.ConnectionString = "Provider=Microsoft.Jet.OLEDB.4.0;Data Source=" & Application.StartupPath & "\bd1.mdb"`
- `conexion.ConnectionString = "Provider=Microsoft.Jet.OLEDB.4.0;Data Source=|DataDirectory|\bd1.mdb"`

Todos los objetos XXXXXConnection generan excepciones del tipo XXXXException que es recomendable tratar.

Para conectarse a una bbdd existente podemos utilizar la opción Conectar con base de datos del menú Herramientas. Seleccionamos el proveedor de nuestra base y la bbdd, Visual Studio genera la conexión, con su correspondiente cadena de conexión y nos permite acceder a las tablas de la base.

Objeto Command

Una vez abierta la conexión se crea un Comando para operar sobre la base, éste puede ser una consulta de selección (lectura de datos) o una consulta de acción (actualización de datos, añadir, borrar, modificar). Una vez establecida la sentencia SQL que queremos que se ejecute llamamos al correspondiente método Execute del comando.

Propiedades

- Commandtext: cadena sql
- Connection: conexión abierta y asociada
- Commandtimeout: análogo a la equivalente de Connection

Métodos

- ExecuteNonQuery: ejecuta una consulta de acción y retorna el nº de filas afectadas
- ExecuteReader: ejecuta consulta de selección y retorna en un objeto DataReader los registros seleccionados
- ExecuteScalar: análogo al anterior pero para valores individuales
- Cancel

NOTA: en las sentencias SQL los números se ponen sin comillas, el texto con comillas simples y las fechas #

Ejemplos

Consultas que solo retornan un valor (campo de la primera fila que cumpla esa selección)

```
String cadenaSQL = "SELECT Nombre FROM empleados WHERE DNI='1111111'"
```

```
MySqlCommand comando = New MySqlCommand(cadena, connexion);  
String nombre=Convert.ToString( comando.ExecuteScalar() );
```

```
cadenaSQL=( "SELECT COUNT (*) FROM Empleados";  
MySqlCommand comando = New MySqlCommand(cadena, connexion);
```

```
int numregistros = Convert.ToInt16( comando. ExecuteScalar());
```

Objeto DataReader

Es el conjunto de registros resultante de una consulta de selección hecha con ExecuteReader.

Propiedades

- IsClosed :
- FieldCount : nº de columnas de los registros
- Item: se le especifica el nombre del campo y nos retorna su valor

```
registros["nombre del campo"]
```

Métodos

- Read: lee un registro y nos retorna true si hay más.
- Close

Es muy importante cerrar este objeto para liberar recursos porque mientras está abierto no se puede ejecutar otro comando con esa conexión, luego hay que liberarla.

Manejo de Parámetros

El comando SQL de un objeto Command puede contener parámetros, para realizar la misma consulta con diferentes valores.

```
"SELECT * FROM Datos WHERE Edad=?"
```

Se crean objetos Parameters, se añaden a la colección Parameter del comando indicando el nombre, el tipo y el tamaño

```
comando.Parameters.Add(new OleDbParameter("@nombre",  
OleDbType.VarChar,10));  
comando.Parameters["@nombre"].Value = TxTnombre.Text;
```

MODO DESCONECTADO

El Objeto **DataSet** representa una base de datos desconectada del proveedor de datos. Almacena tablas y sus relaciones. Es una copia en el cliente de la arquitectura de la base de datos.

El objeto **DataAdapter** es un conector entre el DataSet y la fuente de datos real.

El objeto DataSet dispone a su vez de objetos DataTable que son rellenados por un DataAdapter de tal forma que se puede cerrar la conexión a la bbdd y seguir trabajando con una copia de los datos. Una vez que el usuario ha terminado la edición de los mismos se abre de nuevo la conexión y se utiliza el DataAdapter para enviar de nuevo los datos a la bbdd.

Cada tabla contenida dentro de un objeto DataSet se encuentra disponible a través de su propiedad Tables, que es una colección de objetos System.Data.DataTable. Cada objeto DataTable contiene una colección de objetos DataRow que representan las filas de la tabla. Cada objeto DataRow posee una colección de objetos DataColumn, que representan cada una de las columnas de la fila actual. Además también existen colecciones y objetos para representar las relaciones, claves y valores por defecto existentes dentro de un objeto DataSet.

Para cada objeto DataTable existe una propiedad llamada DefaultView que devuelve un objeto de la clase DataView, que nos ofrece una vista de los datos de la tabla para que podamos recorrer los datos, filtrarlos, ordenarlos, etc.

El objeto DataSet es el más complejo de los objetos que ofrece ADO .NET

Los pasos a seguir para trabajar así son:

1. Generamos un objeto DataAdapter, al que le pasamos la consulta SQL. Para volcar los datos utilizamos su método Fill. Este método recibe dos parámetros, el nombre del DataTable que se creará para guardar la copia de los datos y el DataSet en el que se almacenará ese DataTable.
2. Una vez conectado el DataAdapter podemos extraer los datos que nos interesen y volcarlos al DataSet para trabajar con ellos.

Una vez que tengo las tablas de la bbdd volcadas a la colección Tables del DataSet, trabajo con ésta como con cualquier otra colección:

```
Ds.Tables["Datos"]
```

Cada tabla a su vez dispone de su colección de filas Rows

```
Ds.Tables["Datos"].Rows[índice]
```

A partir de una fila yo puedo acceder a un campo

```
Ds.Tables["nombre"].Rows[fila]["nombre del campo"]
```

Las consultas que se copian pueden provenir de varias tablas.

Podemos utilizar el objeto DataAdapter para realizar cambios en la bbdd. Se hacen sobre la copia y se actualizan después en la bbdd.

Añadir un registro

Habría que indicar al DataAdapter los comandos sql para añadir,.... Esto lo hace de forma automática un objeto OleDbCommandBuilder. Nosotros nos limitamos a añadir una nueva fila a la tabla e indicar el valor de cada campo.

Borrar un Registro

```
Ds.Tables["Datos"].Rows[nº fila].Delete()  
Conexión.open  
Da.Update(Ds,"Datos")  
Conexión.close
```

Modificar un Registro

```
ds.Tables["tabla"].Rows[filaactual]["columna"] = TextBox1.Text  
conexion.Open()  
conector.Update(ds, "direcciones")  
conexion.Close()
```

ORÍGENES DE DATOS

Es un objeto de la aplicación que representa un conjunto de datos almacenados en algún punto, generalmente una bbdd. Nos facilita el acceso y edición de los mismos.

Menú datos: orígenes de datos

Una vez creado se genera un esquema (xsd) con la estructura, además de haberse generado el correspondiente DataSet

En el fichero app.config se almacena la configuración de la cadena de conexión (archivo xml)

Al compilar el proyecto se genera un archivo <NombreDeLaAplicación>.exe.config que guarda dicha configuración. Se puede modificar para cambiar la ruta de la bbdd

Visual Studio dispone de un diseñador para estos orígenes de datos

ENLACE DE DATOS (DATABINDING)

Permite vincular un control a un origen de datos, para que éste muestre automáticamente los datos sin código.

Tenemos la siguiente clasificación en base a

Actualización:

- Enlaces unidireccionales: enlace en modo lectura
- Enlaces bidireccionales: enlace en modo lectura/escritura

Cantidad Propiedades

- Enlaces simples: una propiedad por control (textbox)
- Enlaces complejos: varios campos (gridView)

Para tener controles enlazados, tendremos que manejar los siguientes objetos

- Objetos Binding: enlaza una propiedad de un control con un campo del origen de datos. Este objeto es el que crea el enlace.
- Objeto DataBinding: colección de enlaces
- Objeto BindingManagerBase: reúne todos los binding asociados a un mismo origen de datos. Cuando su posición (propiedad position) cambia, este objeto notifica a todos los objetos binding que deben transmitir al origen de datos los valores actualizados de los controles y obtener los valores de la siguiente posición

Para saber que controles perteneces a su grupo el formulario tiene una propiedad:

- Propiedad BindingContext: la utilizaremos para cuando un formulario tenga varios controles asociados a distintos orígenes.

DATABINDING EN ADO.NET

Podemos generar los objetos binding utilizando un asistente.